



BT4222 Final Report

Movies Recommendation System

GROUP 20

Member	Matric Number
Ng Zheng Leng Bryan	A0273387R
Heng Hong Quan	A0273589J
Lim Jun Ying	A0235191J
Brandon Mok	A0272976L
Tan Ee Sean	A0251917B

Abstract

With the emergence of numerous digital streaming platforms, users are increasingly overwhelmed by the vast array of content choices. In this competitive landscape, content libraries and pricing alone no longer drive user retention. Instead, the quality and personalization of the recommendation engine has become a critical differentiator for streaming services.

This project addresses the business challenge of enhancing user engagement and retention by delivering personalized movie recommendations based on individual viewing histories and movie metadata. We framed this as a user engagement problem, where the goal is to suggest relevant movies based on a user's past interactions, preferences, and movie characteristics. To tackle this, we explored a diverse range of machine learning techniques, including:

We experimented with multiple machine learning techniques, including content based filtering, collaborative filtering, Neural Collaborative Filtering (NCF), Neural Matrix Factorization (NeuMF), and Recurrent Neural Networks (RNNs). Our iterative process involved comparing traditional recommendation techniques with more advanced deep learning models. Through these models, we aimed to evaluate the trade-offs between accuracy, interpretability, and the ability to generalize to unseen user-movie pairs.

Key contributions of this project include:

- Implemented a sequence-aware LSTM model to capture evolving user preferences.
- Integrated additional movie-level features (e.g., vote average, runtime category) into the model pipeline.
- Developed an evaluation framework using Accuracy, NDCG, Precision@K, Recall@K, and Cosine Similarity to assess both ID-level and semantic prediction quality.
- Generated human-readable top-k recommendations mapped to movie titles for interpretability.

This solution offers a scalable and interpretable approach to personalized recommendations and can serve as a foundational architecture for content discovery systems in streaming platforms.

Section 1: Feature Engineering & Textual Enhancement

To enhance the model's ability to learn meaningful patterns from both users and movie characteristics, we began with several feature engineering steps. One key addition was the inclusion of runtime-based categories, where each movie was labeled as "Short", "Medium", or "Long" based on its duration. These were then one-hot encoded to help the model infer user preferences related to pacing and format. Next, we integrated the `vote_average` (i.e., the average rating a movie received) and `vote_count` (i.e., the number of votes a movie received) as a normalized numeric feature to provide a proxy for overall movie quality. Lastly, we also extracted the release year from each movie's `release_date`, enabling us to introduce temporal categorization which would be a useful proxy for generational trends, cinematic styles, or recency bias. These features were concatenated with other inputs to enrich the model's contextual understanding.

For the hybrid model incorporating Content Based Filtering and Collaborative Filtering, we introduce 4 features to provide insights to rating behaviour. The first feature, `rating_diversity`, tallies the number of unique genres of movies that a user has rated. This can provide an indication of the variations in movie preferences - a user with high rating diversity likely has interest in a wide variety of content, whereas a user with a smaller rating diversity may prefer movies of specific genres. The `number_of_ratings_per_user` feature is created by grouping the rows by the `userId` and counting the number of ratings a user has given. A larger value would indicate high engagement in content, whereas a smaller value would imply limited interaction. The third feature, `average_rating`, is calculated by summing up all the rating scores provided by the user and dividing the value by the number of ratings given. K-means clustering is applied to these 3 features to create 2 clusters of users, and the cluster labels are stored in the `engagement_cluster` feature. This feature is used to determine which recommendation technique should be used for each type of user. A value of 0 indicates that a user belongs to the cluster of low engagement, and should hence be recommended movies using the Content Based Filtering technique. On the other hand, a value of 1 indicates that a user is highly engaged and should be recommended movies using the Collaborative Filtering technique.

Beyond structured features, we introduced a layer of semantic enrichment using textual metadata to represent deeper relationships between movies. A new composite field, `Keyword_Genres`, was created by combining each movie's keywords and genres into a single sequence of tokens. These sequences were treated as text documents and used to generate dense vector representations in two ways: through a custom-trained Word2Vec model, which captured co-occurrence patterns among tags, and through pre-trained BERT-based embeddings, which leveraged contextual language understanding to encode

semantic meaning. These embeddings allowed the model to understand nuanced similarities between movies — such as how “space adventure” and “sci-fi thriller” might be conceptually related — even in the absence of explicit user overlap. By incorporating these textual embeddings into our recommendation pipeline, we enabled the model to generalize beyond seen interactions and provide content-aware and semantically informed suggestions.

Section 2: Models and Performance

2.1 Models

2.1.1 Collaborative Filtering (CF)

Collaborative Filtering (CF) is a recommendation technique used to provide recommendations to a user based on the interactions of other users that have similar preferences. We implemented 3 variations of CF models to identify the most suitable CF approach to recommend movies based on user interactions in the form of ratings:

- **User-Based CF** model capturing user preferences based on past ratings.
- **Item-Based CF** model that determines similarity between movies based on ratings provided from users.
- **Hybrid CF** model that makes use of similarity scores computed from both user-based and item-based CF models. We assign different weightages of prediction scores from each model to evaluate whether the hybrid model performs better than the user-based and item-based CF models at a certain set of weightages.

2.1.2 Content Based Filtering (CBF)

Movies can be considered similar to each other when they share a similar context. This can be derived from some of the features in the `movies_metadata` dataset:

- Genres and keywords associated with the movie
- Spoken languages used and countries where the movie is produced
- Overview description of the movie
- Title of the movie as some movies may be part of a franchise series

To recommend movies with similar context, we create a new feature called `textual_representation` to fit the features mentioned above into a single sentence that represents the context of the movie as a whole. A sentence transformer model is used to gain as much semantic meaning as possible by encoding the sentences into embeddings. Movies can then be recommended based on the similarity between their contexts against the context of a movie that has been watched, using similarity scores as a basis to determine how relevant a movie is in terms of the semantic meaning.

2.1.3 Hybrid Model (CBF + CF)

Through exploratory data analysis of user ratings with the use of Principal Component Analysis (PCA), 2 groups of users can be observed. The first group of users is observed to have rated movies from a wide variety of genres and provided many ratings in general, suggesting that these users are highly engaged in watching movies and providing feedback. The second group of users is observed to have a lower level of engagement as they provided few ratings for a limited variety of movies. Given the difference in rating behaviour, we propose a hybrid recommendation model that can recommend movies based on the user's level of engagement. Whilst content-based filtering will be able to provide diverse content-based recommendations for low engagement users, high engagement users will benefit from recommendations provided through collaborative filtering based on ratings given by similar users.

2.1.4 Neural Collaborative Filtering (NCF)

Neural Collaborative Filtering (NCF) is a deep learning framework for recommendation systems that models user-item interactions through neural networks rather than relying on traditional dot-product similarities. Compared to collaborative filtering techniques, such as matrix factorization which computes the interaction score by taking the inner product of user and item embeddings, NCF generalizes this interaction by feeding the embeddings through concatenation into a Multi-Layer Perceptron (MLP). This enables the model to capture non-linear and higher-order interaction patterns.

We have implemented two variants of NCF:

- A **regression model**, trained using **Mean Squared Error** to predict continuous user ratings
- A **binary classification model**, trained using **Binary Cross-Entropy** to predict the likeness of the movie (like or dislike).

2.1.5 Neural Matrix Factorisation (NeuMF)

NeuMF (Neural Matrix Factorization) extends NCF by combining two complementary components: Generalized Matrix Factorization (GMF), which captures linear interactions via element-wise multiplication of user and item embeddings, and MLP, which models non-linear relationships through concatenation and feedforward layers. Both outputs are then concatenated and passed through a final linear layer to produce the predicted score.

We have implemented three variants of NeuMF:

- **Standard NeuMF** model trained using MSE Loss for continuous rating prediction.
- **Hybrid NeuMF via Concatenation** which incorporates movie metadata features such as BERT-based textual embeddings, genre, popularity, and runtime. These movie metadata features are then concatenated with the user–item embeddings in the MLP branch.
- **Hybrid NeuMF via Gated Fusion**. We introduce a gating mechanism that learns to dynamically weight the contribution of movie features during training. The gating mechanism computes a gate value $g \in (0,1)$ by applying a sigmoid activation to a linear layer over the movie features embedding. The MLP output is then scaled by this gate, which allows the model to learn how much emphasis to place on content features dynamically.

These hybrid variants allow NeuMF to be enriched by content-based features that can help alleviate cold-start scenarios when a user interacts with very few movies or when a movie has very few ratings.

2.1.6 Long Short-Term Memory (LSTM)

To effectively capture temporal patterns in user watch behavior, we developed a recommendation model based on the Long Short-Term Memory (LSTM) architecture. LSTMs are a type of recurrent neural network (RNN) designed to retain and learn long-term dependencies in sequential data, making them well-suited for modeling users’ evolving preferences over time. In our context, a user’s watch history is represented as a sequence of movie embeddings, which the LSTM processes to learn an informative user representation. This enables the model to predict or rank future movies based on past consumption patterns.

LSTM with Softmax Classification

In the first variation, we framed the recommendation task as a multi-class classification problem. Using a sliding window approach, we extracted fixed-length sequences from each user’s movie history. These sequences were input into the LSTM, and the final hidden state was passed through a feedforward layer with a softmax output to predict the next movie the user would watch. This method allows the model to directly learn from supervised labels and provides an efficient training and evaluation pipeline using standard classification metrics.

LSTM with Triplet Margin Loss

We implemented a second variation using triplet margin loss. Instead of predicting a fixed label, this approach learns a ranking-based embedding space. Each training instance consists of an anchor (a sequence representing a user’s recent history), a positive (the actual next movie), and a negative (a random movie not watched by the user). The model learns to bring the user embedding closer to the positive and push it further from the negative by a specified margin. This enables generalization to the full movie space, allowing the model to recommend semantically similar and contextually relevant movies—even those never seen during training. This ranking-based formulation aligns better with real-world recommendation systems, where the goal is not to predict a single correct item, but to retrieve a ranked list of relevant content from a large catalog.

2.2 Models Evaluation and Performance

	Model Performance									
Evaluation Metrics k = 10	CBF	CF	Hybrid (CBF + CF)	NCF (Regression)	NCF (Binary)	NeuMF	NeuMF + Movies Data	NeuMF + Movies Data via Gated Fusion	LSTM (SoftMax)	LSTM (Triplet Margin Loss)
Accuracy	NA	NA	NA	0.5872	0.7876	0.5716	0.5964	0.6253	0.9103	0.0527
		RMSE: 3.2235		0.5740	0.6590	0.5839	0.5953	0.6106	0.8447	0.0519
NDCG@k	0.9469	NA	0.9431	0.8055	0.8873	0.7810	0.8030	0.8054	0.8746	0.6757
		0.9227		0.8335	0.7514	0.8393	0.8458	0.8404	0.8190	0.6674
Precision@k	0.8004	NA	0.8918	0.4423	0.8483	0.3981	0.4689	0.5256	0.2855	0.4922
		0.6885		0.1528	0.4915	0.1431	0.1683	0.2098	0.2660	0.4776
Recall@k	0.4391	NA	0.2146	0.2229	0.6116	0.1987	0.2379	0.2730	0.9937	0.8467
		0.8173		0.2381	0.9281	0.2284	0.2667	0.3288	0.9515	0.8469
F1 Score @K	0.5671	NA	0.3460	0.2473	0.5947	0.2242	0.2690	0.3078	0.4435	0.6226
		0.7474		0.1548	0.5501	0.1454	0.1736	0.2162	0.4158	0.6108

Orange cells and red cells represent performance scores for **train set and **test set** respectively

Section 3: Contribution and Justification

Contribution		Level 1	Level 2	Level 3
Use valuable and high-quality datasets, including integrating existing datasets, crawling or retrieving data, etc	Effort		☑	
	Effectiveness			☑
Level 1: Use existing datasets acquired directly from external resources Level 2 - 3: collecting new data or integrate datasets from multiple sources considering the richness of the information, the amount of data points, the creativity in finding surprisingly useful resources or the intelligence in integrating datasets for new insights				
Creativity in feature engineering	Effort			☑
	Effectiveness			☑
Level 1: generate new features based on descriptive statistics (e.g., mean, variance, min/max etc.) or cross featuring Level 2-3: generate new features by applying relevant theories, domain knowledge, representative learning, social network analysis, or other machine learning or econometrics methods. Adopting methods proposed by high-quality research papers is also encouraged.				
Design or adaptation of new ML methods/architecture or the integration of existing methods with a balance of resource and cost	Effort			☑
	Effectiveness			☑
Level 1: ensemble learning by straightforwardly integrating multiple ML models Level 2-3: Design innovative architecture or pipeline or adapt ML methods, which have changed the way how learning and prediction will be conducted. Adopting methods proposed by high-quality research papers is also encouraged. Tips: To design innovative architecture or pipeline does not necessarily mean sophisticated algorithm design. Very often, you may achieve surprising results by changing how data is sampled and fed into models, how embedding is applied, or how multiple models' inputs and outputs are integrated or positioned in a pipeline.				
Creativity and insights in understanding or further explaining the ML output and performance, in identifying the bias of machine learning output against the real distribution, the business implications of applying it into practice	Effort		☑	

	Effectiveness		☒	
Level 1: describe results and explain the model performance without support by quantitative analysis Level 2-3: Use data analytics methods (econometrics, network analysis, etc. what you have learned from the other courses) and additional ML methods to clarify, distinguish, identify the bias of, or evaluate your ML output towards effective business decision making.				

3.1 Dataset Refinement

In building our recommendation system, we focused not just on model architecture, but also on curating a high-quality dataset that would serve as a strong foundation for learning meaningful user preferences. Rather than relying on a single file or off-the-shelf format, we combined and enhanced three datasets to create a more robust and semantically rich resource for training and evaluation. This refinement allowed us to capture not just what users watch, but also the underlying themes, content features, and metadata that influence viewing behavior.

3.1.1 Integration of Multi-Source Movie Data

Our dataset is based on the publicly available "The Movies" dataset from Kaggle, and it integrates three key files: **movies_metadata.csv**, **keywords.csv**, and **ratings_small.csv**. Each file served a distinct purpose and added a unique layer of information to our final dataset:

- **movies_metadata.csv** provided a broad range of movie-level attributes such as release date, runtime, genres, popularity, and movie ratings.
- **keywords.csv** contributed semantically rich tags, describing conceptual themes behind each movie. For movies without keywords, we supplemented them with genre information to preserve semantic consistency.
- **ratings_small.csv** contained a subset of 100,000 user ratings, giving us just enough diversity and behavioral data to build a proof-of-concept model while keeping processing efficient.

Together, these files formed a unified and enriched dataset that combined structured metadata, user-generated feedback, and semantic descriptors of content.

3.1.2 Data Cleaning and Standardisation

To ensure that our model was trained on consistent and relevant data, we conducted multiple cleaning steps. We removed irrelevant columns such as **poster_path**, **homepage**, and **video**, which offered little predictive value. Fields with overwhelming missing data—like **budget**, **revenue**, and **status**—were also dropped. We filtered to include only movies that had been officially released.

We addressed missing values in critical fields like **runtime**, **vote_average**, and **release_date** by dropping incomplete entries. We also ensured that nested fields (such as genres, languages, and production details) were flattened and reformatted to allow smoother processing downstream.

3.1.3 Unified Representation through Merging

To create our final modeling-ready movie dataset, we merged **movies_metadata.csv** with **keywords.csv** using the shared **id** field. This join ensured that each movie was enriched with both structured metadata and semantically meaningful keywords. In cases where keyword information was missing, we filled in the gaps using genre data from the metadata to maintain semantic consistency. This combined dataset served as the core input for all content-based components of our recommendation system.

Meanwhile, **ratings_small.csv** was retained as a separate source of user interaction data. It was selectively merged during downstream modeling tasks to extract personalised user watch histories and construct training samples. This modular separation of movie metadata and user interactions allowed us to support both content-based and hybrid recommendation architectures flexibly.

3.2 Feature Engineering

#	Approach (in order)	Purpose
1	vote_average_norm	Represents the quality of a movie as perceived by the general audience, normalized for scale compatibility. Helps the model learn preference for highly-rated content.
2	vote_count_norm	Reflects the popularity or social proof of a movie. Normalized to balance influence between niche and mainstream movies.
3	release_year_norm	Indicates the recency of a movie's release on a normalized scale, allowing the model to learn user preference trends for older vs. newer movies.
4	runtime_category	Categorical bucketing of movie length (e.g., short, medium, long). Helps model capture user preferences for movie duration.
5	years_since_release	Number of years since released from 2017 as the dataset is up till 2017.
6	keyword_values	Captures semantic content or themes of the movie. Enables content-based recommendations through similarity in topic/genre.
7	average_rating	Average score of ratings provided by the user.
8	rating_diversity	Indicates the number of unique genres this user has watched, which shows their engagement to variety.
9	number_of_ratings_per_user	Indicates how active a user is. Useful for personalizing recommendation strength based on engagement level.

10	engagement_cluster	Captured by combining rating_diversity (unique genres rated), average_rating and number_of_ratings_per_user using K-means clustering.
11	textual_representation	Fit genres, keywords, spoken languages, countries, overview and title into a single sentence that represents the context of the movie as a whole
12	Word2Vec	Generates dense semantic embeddings of movie content (e.g., genres, keywords). Enables similarity-based reasoning across movies beyond explicit categories.
13	Sentence Transformer	Generates dense semantic embeddings of the textual representation of movie content.

3.3 Design or Adaption of New ML Methods/Architecture

3.3.1 Hybrid NeuMF with Movies Features Embedding

This hybrid model extends the NeuMF by incorporating movie metadata into the recommendation process. In this model, movie content (BERT-based textual embeddings, years since release, runtime category, vote statistics, and popularity) is preprocessed and combined into a high-dimensional feature vector (391 dimensions). This vector is then projected via a fully connected layer into a 64-dimensional embedding.

By incorporating movie features directly into the NeuMF architecture via simple concatenation, the model gains access to rich, content-based information that is often absent in purely collaborative approaches. Traditional collaborative filtering methods rely solely on historical user–item interactions, which can be sparse or biased toward popular items. Integrating external movie metadata, such as BERT-based textual embeddings, provides the model with additional context. This enriched feature space allows the model to distinguish between movies not just by latent interaction signals but also by their intrinsic content characteristics. Consequently, the model is better able to generate personalised recommendations for users

who have limited historical data, commonly known as the cold-start problem. Studies have shown that hybrid recommender systems that blend collaborative and content-based methods can significantly enhance the diversity and relevance of recommendations (He et al., 2017; Zhang et al., 2019).

3.3.2 Hybrid NeuMF with Gated Fusion Layer

In this variant, the integration of movie features is enhanced by a dynamic gating mechanism. This model is built on the same foundation as the previous hybrid variant, but instead of simply concatenating the content features, it uses a gating layer to adaptively control their influence in the prediction.

Architecture Details:

- **User and Item Embeddings:**

Users and movies are embedded into 32-dimensional vectors.

- **Movie Feature Transformation:**

The 391-dimensional movie feature vector is transformed into a 64-dimensional representation using a linear layer with ReLU activation.

- **MLP Branch with Gating:**

The MLP branch receives the concatenated vector of user embedding (32-dim), item embedding (32-dim), and movie feature embedding (64-dim) — forming a 128-dimensional vector. It processes this vector through:

- A fully connected layer ($128 \rightarrow 128$) with ReLU activation,
- A second fully connected layer ($128 \rightarrow 64$) with ReLU,
- A dropout layer ($p=0.5$) for regularization.

- In parallel, the movie feature embedding is fed into a dedicated **gating layer**, which is a linear layer followed by a sigmoid activation. This layer computes a scalar gate weight (between 0 and 1) that dynamically scales the MLP output.

- **Fusion and Final Prediction:**

The GMF branch (element-wise multiplication of user and item embeddings, 32-dim) is concatenated with the gated MLP output (64-dim), forming a 96-dimensional vector. This fused representation is then passed through a final linear output layer to produce a single scalar prediction.

The gated fusion variant takes the idea further by not only incorporating movie features but also dynamically determining how much those features should influence the final prediction. In many cases, the quality or relevance of content-based features can vary greatly across different users or items. For example, some users may exhibit strong collaborative signals, rendering the additional movie metadata less critical, while for others—especially in cold-start situations—the movie features may be the only reliable indicator of preference.

A **gating mechanism** is introduced to address this variability. The gating layer learns, for each input sample, a weight (or gate) that scales the contribution of the movie feature embedding before it is combined with the collaborative filtering (GMF) output. This adaptive mechanism allows the model to emphasize movie content when it is informative and discount it when collaborative signals are strong. Such dynamic weighting leads to better overall performance because it tailors the information fusion based on the contextual relevance of movie features, allowing for more precise recommendations. Research in adaptive deep models and attention mechanisms (Ma et al., 2019; Wang et al., 2021) supports the idea that gating mechanisms improve the model’s ability to learn robust representations, particularly in multi-source data fusion scenarios.

3.3.3 Overcoming Softmax Limitations with Triplet Margin Loss in LSTM

Although the classification-based model performed well on our dataset, it revealed a critical limitation: The softmax layer can only assign probabilities to movies present in the training label set. In our case, the training data included around 900 unique movies, whereas the full movie database contains over 45,000 titles. This means the model could not recommend unseen or infrequent movies, limiting its generalisation to the broader catalog.

To overcome this limitation, we adapted our architecture by replacing the classification head with a distance-based ranking objective using triplet margin loss. Instead of predicting a single movie label, the model now learns to embed user behavior and movie representations into the same latent space, where relevant movies are positioned closer to the user vector and irrelevant ones are pushed farther away.

This adjustment enables the model to perform recommendations across the entire movie database, not just the movies seen during training. By structuring training samples as (anchor, positive, negative) triplets—where the anchor is a user’s recent watch sequence, the positive is the next actual movie watched, and the negatives are random movies not watched by the user—the model learns a generalisable similarity function. This approach makes it inherently more scalable and suitable for cold-start or long-tail movie recommendations.

Architecture Details:

Sequential Input Encoding (LSTM)

The model receives a sequence of movie feature vectors representing a user's watch history. These feature vectors are a concatenation of:

- **Word2Vec-based semantic embeddings** from Genres + Keywords
- **Normalized numerical features:** `vote_average_norm`, `vote_count_norm`, and `release_year_norm`

The sequence is passed through a single-layer LSTM:

- Input dimension: 103 (dimensionality of each movie vector + Normalized numerical features)
- Hidden dimension: 64

The LSTM outputs a final hidden state representing the **user embedding**.

Categorical Embeddings

Two categorical features—`production_company` and `runtime_category`—are encoded using:

- Two separate embedding layers of size 16
- These embeddings are concatenated with the LSTM output to enhance the user vector with metadata context.

Feedforward Projection

The combined vector (LSTM output + categorical embeddings) is passed through:

- A dropout layer ($p = 0.3$)
- A ReLU-activated linear layer (hidden $\rightarrow 128$)
- A second linear layer projecting to a final embedding space (output dim = 103)

Triplet Loss Objective

During training, the model receives triplets:

- **Anchor:** user watch history sequence
- **Positive:** vector of the next movie the user actually watched
- **Negative:** vector of a random movie not watched by the user

The model is trained to minimize the distance between the anchor and the positive, while maximizing the distance from the negative by at least a margin of $m = 0.3$, where m is a hyperparameter that defines the minimum distance by which the negative sample should be farther away from the anchor than the positive sample. This encourages the model to learn meaningful proximity relationships in the embedding space. Recent work by Wang et al. (2024) supports the effectiveness of triplet contrastive loss in enhancing

recommendation performance by learning more discriminative and generalisable sequence representations in embedding-based ranking tasks.

3.4 Understanding Prediction Results to Resolve Gaps Between ML Outcomes and Business Decision-Making

3.4.1. Choice of Evaluation Metrics & k-value (explain about why we choose those evaluation metrics)

Our team selected Accuracy, NDCG@10, Precision@10, Recall@10, and F1-score@10 as our evaluation metrics.

These metrics are widely adopted in recommendation system research and allow for more standardized comparison with existing works. In particular, Precision, Recall, and F1-score remain among the most frequently used metrics, which supports their inclusion. NDCG@10 was chosen for its ability to account for the position of relevant items in the recommendation list, offering a more nuanced evaluation of ranking quality.

We adopted $k=10$ as our cutoff point, aligning with established norms in the field. Using a top-10 recommendation list provides a practical balance, as it reflects real-world usage where users are typically shown a limited number of options and supports engagement by not overwhelming users with too many choices. Evaluating at $k=10$ also ensures sufficient granularity in measuring ranking effectiveness while still capturing relevance in a concise, user-centric way.

3.4.2. Interpreting Prediction Results

When evaluating the performance of the various recommendation models, it is essential to interpret the results not only from a technical perspective but also in terms of how they align with business goals, such as enhancing user satisfaction, increasing engagement, and improving retention.

- For instance, the Content-Based Filtering (CBF) model shows strong performance in ranking metrics, with an NDCG@ k of 0.9469 and Precision@ k of 0.8004, indicating its ability to recommend relevant items based on user preferences. However, it suffers from a relatively low Recall@ k of 0.4846, suggesting that it may miss a considerable portion of potentially interesting items. This is typical of content-based systems, which are particularly useful for new users or items (cold-start scenarios) but often lack diversity and fail to capture broader user behavior

patterns.

- The Neural Collaborative Filtering (NCF) models present a more nuanced comparison. The binary classification variant outperforms its regression-based counterpart across nearly every metric. Binary NCF achieves an accuracy of 0.7876, a high NDCG@k of 0.8873, and a strong Precision@k of 0.8483. Its Recall@k of 0.6116 and F1 Score@k of 0.5501 indicate that it balances hit rate and precision effectively, making it a promising candidate for deployment in large-scale recommendation engines. In contrast, the regression variant, while still competitive (Accuracy: 0.5872), lags in ranking and recall metrics.
- NeuMF, a hybrid deep learning model combining matrix factorization with neural networks, further improves performance, particularly in accuracy (0.5964) and precision (0.4689), though it doesn't surpass binary NCF. Adding movie metadata to NeuMF slightly boosts performance (Accuracy: 0.6253, Precision@k: 0.5256), suggesting that content enrichment improves user relevance. Using gated fusion with metadata offers further gains in ranking metrics like NDCG@k (0.9245), demonstrating that combining structured and unstructured data helps enhance personalisation.
- The LSTM (Triplet Margin Loss) model shows a very low accuracy (0.0527), which may imply poor performance. However, this model is specifically optimised for ranking tasks rather than classification. Its relatively high Recall@k (0.8399) and decent NDCG@k (0.6757) validate its value in ranking relevant items at the top of recommendation lists. Such models are particularly useful in settings where top-N recommendations are more important than predicting exact movies.

In summary, while some models like binary NCF and NeuMF with metadata deliver strong overall results and are likely to contribute positively to user engagement and retention, others like LSTM Triplet-Loss offer more context-aware and rank-sensitive solutions. These nuanced interpretations are vital in aligning machine learning outcomes with business goals, where improving user experience and minimising decision fatigue are as crucial as the technical performance itself.

However, a crucial step in deploying machine learning (ML) models in production is ensuring that model outputs translate into tangible business outcomes. While technical metrics such as Accuracy, Precision@k, Recall@k, NDCG@k, and F1 Score@k are vital for benchmarking model performance, they do not always align perfectly with business value or user satisfaction.

Ultimately, the best model should be chosen not just based on metric superiority but on alignment with business KPIs such as increased watch time, reduced user churn, and higher conversion from free to paid users.

Section 4: References

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T. S. (2017). Neural Collaborative Filtering. <https://doi.org/10.1145/3038912.3052569>

Ma, J., Zhao, P., Yi, X., Chen, Q., Hong, L., & Chi, E. H. (2019). Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts <https://doi.org/10.1145/3308558.3313710>

Xia, H., Wang, Z., Du, B., Zhang, L., Chen, S., & Chun, G. (2019). Leveraging ratings and reviews with gating mechanism for recommendation. Proceedings of the 28th ACM International Conference on Information and Knowledge Management, 1573–1582. <https://doi.org/10.1145/3357384.3357919>

Zhang, S., Yao, L., Sun, A., & Tay, Y. (2019). Deep Learning Based Recommender System: A Survey and New Perspectives. *ACM Computing Surveys (CSUR)*, 52(1), 1–38. <https://doi.org/10.1145/3285029>

Wang, W., Liu, X., Zhuang, F., & He, Q. (2024). *Learnable sequence augementer for triplet contrastive learning in sequential recommendation*. <https://doi.org/10.48550/arXiv.2503.20232>